Tema 3 — Construcción de Modelos (Supervisado)
Certificación Profesional en Ciencia de Datos (IBM)

Índice

1. Objetivo del Tema 3 y pipeline estándar **3**
- 1.1. Pipeline general en `scikit-learn` . . . 3
- 1.2. Plantilla base reutilizable . . . 3

2. Árboles de Decisión (Clasificación) **4**
- 2.1. Qué hace el método (intuición y mecánica) . . . 4
- 2.2. Cómo decide un split: entropía / gini . . . 4
- 2.3. Overfitting y control de complejidad . . . 4
- 2.4. Código del método (plantilla y explicación) . . . 4
  - 2.4.1. Preprocesado típico del lab . . . 4
  - 2.4.2. Entrenamiento y evaluación . . . 5
  - 2.4.3. Visualización e interpretación . . . 5
- 2.5. Uso real (empresa) . . . 5

3. Árboles de Regresión **5**
- 3.1. Qué hace el método . . . 5
- 3.2. Cómo decide splits: minimizar error cuadrático . . . 6
- 3.3. Métricas correctas . . . 6
- 3.4. Código (plantilla del lab) . . . 6
- 3.5. Notas prácticas . . . 6

4. Clasificación multiclase (estrategias OvR y OvO) **6**
- 4.1. Qué es multiclase . . . 6
- 4.2. Por qué hablamos de "estrategias" . . . 7
- 4.3. Importante (conexión con tu duda anterior) . . . 7
- 4.4. Preprocesado típico del dataset de obesidad . . . 7
- 4.5. Código: OvR y OvO (estructura) . . . 7
- 4.6. Cuándo OvR vs OvO . . . 8

5. K-Nearest Neighbors (KNN) — Clasificación **8**
- 5.1. Qué hace el método . . . 8
- 5.2. Por qué el escalado es crítico . . . 8
- 5.3. Código del lab y explicación . . . 8
- 5.4. Búsqueda del mejor K (parte práctica) . . . 8

6. Support Vector Machines (SVM) — Clasificación **9**
- 6.1. Qué hace el método . . . 9
- 6.2. Por qué el escalado importa . . . 9
- 6.3. Multiclase con SVM . . . 9
- 6.4. Código típico (SVM lineal) . . . 9

7. Caso aplicado: Fraude con tarjeta (Árbol vs SVM) y datos desbalanceados **10**
7.1. Por qué este notebook es distinto . . . 10
7.2. Métrica clave: ROC-AUC . . . 10
7.3. Árbol con `sample_weight` . . . 10
7.4. SVM con `class_weight` y `decision_function` . . . 10
7.5. Preprocesado típico del dataset de fraude . . . 10

8. Ensembles: Random Forest (bagging) y XGBoost (boosting) **11**
8.1. Por qué ensembles . . . 11
8.2. Random Forest (Bagging) . . . 11
8.2.1. Cómo funciona . . . 11
8.2.2. Código y parámetros clave . . . 11
8.3. XGBoost (Boosting) . . . 11
8.3.1. Cómo funciona . . . 11
8.3.2. Código (estructura típica) . . . 11
8.4. Random Forest vs XGBoost (decisión práctica) . . . 12

9. Comparativa global: cuándo usar cada método **12**
9.1. Regla rápida de elección . . . 12

10. Mini-chuleta de funciones (sklearn) para examen **12**
10.1. Separación train/test . . . 12
10.2. Entrenar, predecir y evaluar . . . 13
10.3. ROC-AUC . . . 13

1. Objetivo del Tema 3 y pipeline estándar

En aprendizaje supervisado, tenemos datos (X, y) donde:

- X son las **variables explicativas** (features).
- y es la **variable objetivo** (target):
  - **Clasificación**: y es una categoría (binaria o multiclase).
  - **Regresión**: y es un número real.

1.1. Pipeline general en `scikit-learn`

1. **Cargar datos** (`pandas.read_csv`).
2. **EDA mínima** (tipos, nulos, distribución de clases, correlaciones orientativas).
3. **Preprocesado**:
   - Codificación de categóricas (one-hot, label encoding dependiendo del caso).
   - Escalado si el método depende de distancias o hiperplanos (KNN, SVM, etc.).
4. **Separar train/test** con `train_test_split`.
5. **Entrenar** (`fit`).
6. **Predecir** (`predict`, `predict_proba` o `decision_function`).
7. **Evaluar** con métricas adecuadas.
8. **Ajustar hiperparámetros** (búsqueda de K, `max_depth`, etc.).

1.2. Plantilla base reutilizable

```python
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

df = pd.read_csv("datos.csv")

# 1) target y features
y = df["target"]
X = df.drop(columns=["target"])

# 2) split reproducible
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y # stratify si clasificacion
)

# 3) modelo (se cambia segun metodo)
model = ...
model.fit(X_train, y_train)

# 4) prediccion + evaluacion
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Notas clave:

- `random_state`: fija el azar para reproducibilidad.
- `stratify=y`: en clasificación mantiene proporciones de clases en train/test.
- En problemas desbalanceados, accuracy puede engañar ⇒ usar ROC-AUC, recall, F1, PR-AUC.

2. Árboles de Decisión (Clasificación)

2.1. Qué hace el método (intuición y mecánica)

Un árbol de decisión de clasificación aprende **reglas** tipo "si-entonces". Divide el espacio de variables mediante **splits**:

$$\text{si } x_j \leq t \Rightarrow \text{rama izquierda; si } x_j > t \Rightarrow \text{rama derecha}$$

En cada nodo, el algoritmo prueba muchas variables x_j y muchos umbrales t y elige el split que **mejor separa las clases**.

2.2. Cómo decide un split: entropía / gini

En el notebook se usa `criterion=.^entropy"`: se elige el split que maximiza la **ganancia de información** (reduce la entropía).

- Alta entropía: clases mezcladas (desorden).
- Baja entropía: nodo "puro" (una clase domina).

2.3. Overfitting y control de complejidad

Un árbol puede memorizar. Para controlarlo:

- `max_depth`: limita profundidad (clave en el lab: comparar 4 vs 3).
- `min_samples_leaf`: hojas con mínimo de muestras (evita reglas basadas en 1-2 casos).
- `min_samples_split`: mínimo para dividir.

2.4. Código del método (plantilla y explicación)

2.4.1. Preprocesado típico del lab

En el lab (drug dataset) hay categóricas (Sexo, BP, Colesterol). Un árbol de sklearn no acepta strings: hay que convertir.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df["Sex"] = le.fit_transform(df["Sex"])
df["BP"] = le.fit_transform(df["BP"])
df["Cholesterol"] = le.fit_transform(df["Cholesterol"])
```

Qué hace cada función:

- `fit_transform`: aprende el mapeo categoría→entero y lo aplica.
- Ojo: `LabelEncoder` introduce un orden artificial (0<1<2). En árboles suele funcionar, pero para ser estrictos con variables nominales, se prefiere **one-hot** (`pd.get_dummies`).

2.4.2. Entrenamiento y evaluación

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

y = df["Drug"]
X = df.drop(columns=["Drug"])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=32
)

tree = DecisionTreeClassifier(
    criterion="entropy", # ganancia de informacion
    max_depth=4, # control de overfitting
    random_state=42
)

tree.fit(X_train, y_train)
y_pred = tree.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

Explicación de los parámetros clave:

- `criterion=.^entropy"`: elige splits reduciendo entropía.
- `max_depth`: si baja, el árbol es más simple (menos varianza, más sesgo).
- `random_state`: reproducible (misma estructura si todo lo demás se mantiene).

2.4.3. Visualización e interpretación

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(14, 8))
plot_tree(tree, filled=True, feature_names=X.columns)
plt.show()
```

Qué aporta: convierte el modelo en reglas interpretables (muy útil en negocio: compliance, explicación al cliente, auditoría).

2.5. Uso real (empresa)

- Crédito: reglas claras para aprobar/rechazar.
- Medicina: decisión interpretable (aunque con cautela y validación).
- Atención al cliente: clasificación de incidencias por reglas.

3. Árboles de Regresión

3.1. Qué hace el método

Mismo esquema de árbol, pero el objetivo y es **numérico**. Cada hoja predice un valor (normalmente la **media** de los y dentro de la hoja).

3.2. Cómo decide splits: minimizar error cuadrático

En regresión, el mejor split es el que reduce más la variabilidad del target dentro de los nodos (criterio `squared_error`). Intuición: queremos que dentro de cada hoja los y sean "parecidos".

3.3. Métricas correctas

- MSE: **Mean Squared Error** (penaliza errores grandes).
- R^2: mejora frente a predecir siempre la media ($0 \approx$ baseline, 1 perfecto).

3.4. Código (plantilla del lab)

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error

df = pd.read_csv("yellow-tripdata.csv")

y = df["tip_amount"].astype("float32")
X = df.drop(columns=["tip_amount"])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

reg_tree = DecisionTreeRegressor(
    criterion="squared_error",
    max_depth=8,
    random_state=35
)

reg_tree.fit(X_train, y_train)
y_pred = reg_tree.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = reg_tree.score(X_test, y_test) # R^2
print("MSE:", mse)
print("R2:", r2)
```

Qué hace `score` en regresión: devuelve R^2.

3.5. Notas prácticas

- Si aumentas `max_depth` suele bajar MSE en train pero puede empeorar en test (overfitting).
- Para mejorar, se usan ensembles (Random Forest, XGBoost).

4. Clasificación multiclase (estrategias OvR y OvO)

4.1. Qué es multiclase

Cuando y tiene 3 o más categorías (ej.: niveles de obesidad).

4.2. Por qué hablamos de "estrategias"

Muchos clasificadores son binarios por naturaleza o se implementan de forma binaria internamente. Para multiclase se usan estrategias:

- **OvR (One-vs-Rest)**: un modelo por clase vs el resto.
- **OvO (One-vs-One)**: un modelo por cada pareja de clases.

4.3. Importante (conexión con tu duda anterior)

En el notebook se aplica esto con un clasificador lineal típico (muy común es usar un modelo logístico, pero aquí lo relevante es la **estrategia multiclase**, no el "nombre" del algoritmo). **La idea**: convertir multiclase en varios problemas binarios y combinar scores/votos.

4.4. Preprocesado típico del dataset de obesidad

- One-hot para categóricas (evita orden artificial).
- Escalado para continuas si se usa un modelo sensible a escala.
- Codificar target a enteros (0..K-1) para entrenar.

4.5. Código: OvR y OvO (estructura)

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.multiclass import OneVsRestClassifier, OneVsOneClassifier

# Ejemplo: base_estimator puede ser cualquier clasificador binario
from sklearn.linear_model import LogisticRegression

df = pd.read_csv("obesity.csv")

y = df["NObeyesdad"].astype("category").cat.codes
X = df.drop(columns=["NObeyesdad"])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

base = LogisticRegression(max_iter=1000)

ovr = OneVsRestClassifier(base)
ovr.fit(X_train, y_train)
pred_ovr = ovr.predict(X_test)
print("OvR accuracy:", accuracy_score(y_test, pred_ovr))

ovo = OneVsOneClassifier(base)
ovo.fit(X_train, y_train)
pred_ovo = ovo.predict(X_test)
print("OvO accuracy:", accuracy_score(y_test, pred_ovo))
```

Qué hace cada parte:

- `OneVsRestClassifier(base)`: crea K modelos (uno por clase).
- `OneVsOneClassifier(base)`: crea $K(K-1)/2$ modelos por pares.
- `max_iter`: evita que el optimizador pare demasiado pronto (convergencia).

4.6. Cuándo OvR vs OvO

- OvR: simple, eficiente, suele funcionar bien.
- OvO: más modelos, a veces mejora si las fronteras entre pares son claras.

5. K-Nearest Neighbors (KNN) — Clasificación

5.1. Qué hace el método

KNN clasifica un punto nuevo mirando sus K vecinos más cercanos en el conjunto de entrenamiento y votando la clase mayoritaria.

5.2. Por qué el escalado es crítico

Como se basa en distancias, si una variable está en euros (0–200) y otra en recuento (0–5), la de euros dominará la distancia. Por eso se usa `StandardScaler`.

5.3. Código del lab y explicación

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

df = pd.read_csv("teleCust1000t.csv")

y = df["custcat"]
X = df.drop(columns=["custcat"])

# Escalado
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=4, stratify=y
)

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
pred = knn.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
```

Qué hace `StandardScaler`:

- `fit`: calcula media y desviación de cada columna.
- `transform`: convierte a escala comparable (media 0, varianza 1).

5.4. Búsqueda del mejor K (parte práctica)

```
Ks = 20
acc = []
```

```
for k in range(1, Ks+1):
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    acc.append(accuracy_score(y_test, pred))

best_k = np.argmax(acc) + 1
print("Mejor K:", best_k, "con accuracy:", max(acc))
```

Interpretación:

- K pequeño: decisiones muy sensibles al ruido (varianza alta).
- K grande: decisiones más suaves, puede perder detalle (sesgo alto).

6. Support Vector Machines (SVM) — Clasificación

6.1. Qué hace el método

SVM busca un hiperplano $w^T x + b = 0$ que separa dos clases maximizando el margen. Predicción:

$$f(x) = w^T x + b, \quad f(x) > 0 \Rightarrow clase\ 1, \quad f(x) < 0 \Rightarrow clase\ 0$$

La magnitud $|f(x)|$ indica cuán lejos estás de la frontera (confianza relativa).

6.2. Por qué el escalado importa

SVM (especialmente lineal) usa combinaciones lineales de variables; si las escalas son muy diferentes, el ajuste se distorsiona. Escalar suele ser obligatorio.

6.3. Multiclase con SVM

SVM es binaria por naturaleza, pero se extiende a multiclase con OvR u OvO (misma lógica explicada antes).

6.4. Código típico (SVM lineal)

```
from sklearn.svm import LinearSVC

svm = LinearSVC(
    class_weight="balanced", # clave si hay desbalance
    random_state=31,
    loss="hinge",
    fit_intercept=False
)

svm.fit(X_train, y_train)

# Prediccin de clase
pred = svm.predict(X_test)
```

Notas:

- `class_weight="balanced"`: ajusta pesos inversamente proporcionales a la frecuencia de clases.
- `decision_function`: devuelve el score $f(x)$ (ideal para ROC-AUC).
- `predict_proba`: no existe en `LinearSVC`. Si se necesitan probabilidades, se calibran aparte.

7. Caso aplicado: Fraude con tarjeta (Árbol vs SVM) y datos desbalanceados

7.1. Por qué este notebook es distinto

Introduce dos ideas prácticas cruciales:

1. **Desbalance extremo**: accuracy puede ser inútil.
2. **Métricas y pesos**: ROC-AUC, `class_weight`, `sample_weight`.

7.2. Métrica clave: ROC-AUC

ROC-AUC mide la capacidad del modelo para ordenar positivos por encima de negativos. En fraude interesa detectar el 1 (fraude) sin disparar falsos positivos.

7.3. Árbol con `sample_weight`

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.utils.class_weight import compute_sample_weight
from sklearn.metrics import roc_auc_score

w_train = compute_sample_weight(class_weight="balanced", y=y_train)

dt = DecisionTreeClassifier(max_depth=4, random_state=35)
dt.fit(X_train, y_train, sample_weight=w_train)

proba = dt.predict_proba(X_test)[:, 1] # probabilidad de fraude
auc = roc_auc_score(y_test, proba)
print("ROC-AUC (DT):", auc)
```

Claves:

- `compute_sample_weight`: da más peso a la clase minoritaria en train.
- `predict_proba` en árboles: devuelve probabilidad por clase; para ROC-AUC usamos la columna del positivo.

7.4. SVM con `class_weight` y `decision_function`

```
from sklearn.svm import LinearSVC
from sklearn.metrics import roc_auc_score

svm = LinearSVC(class_weight="balanced", random_state=31)
svm.fit(X_train, y_train)

scores = svm.decision_function(X_test) # score, no prob
auc = roc_auc_score(y_test, scores)
print("ROC-AUC (SVM):", auc)
```

Por qué vale para ROC-AUC: ROC-AUC solo necesita un *score ordenable* (no probabilidades).

7.5. Preprocesado típico del dataset de fraude

- `StandardScaler`: recomendable para SVM.
- A veces normalización (L1) por fila para homogeneizar magnitudes (depende del diseño del lab).

8. Ensembles: Random Forest (bagging) y XGBoost (boosting)

8.1. Por qué ensembles

Un árbol único es inestable: pequeños cambios en datos cambian estructura. Los ensembles combinan muchos árboles para mejorar generalización.

8.2. Random Forest (Bagging)

8.2.1. Cómo funciona

- Se crean muchos árboles.
- Cada árbol entrena con un **bootstrap** (muestra con reemplazo).
- En cada split se prueba un subconjunto aleatorio de variables.
- En clasificación se vota; en regresión se promedia.

8.2.2. Código y parámetros clave

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

rf = RandomForestClassifier(
    n_estimators=200, # numero de arboles
    max_depth=None, # profundidad (None = crece libre)
    max_features="sqrt", # aleatoriza features en splits
    random_state=42,
    n_jobs=-1 # usa todos los cores
)

rf.fit(X_train, y_train)
pred = rf.predict(X_test)
print("Accuracy RF:", accuracy_score(y_test, pred))
```

Qué significa cada parámetro:

- `n_estimators`: más árboles suele mejorar estabilidad (hasta saturar).
- `max_features`: aumenta diversidad de árboles (reduce correlación).
- `n_jobs=-1`: acelera entrenando en paralelo.

8.3. XGBoost (Boosting)

8.3.1. Cómo funciona

A diferencia de Random Forest, los árboles se entrenan **en secuencia**. Cada nuevo árbol corrige errores del conjunto anterior. Esto suele aumentar potencia predictiva en tabular.

8.3.2. Código (estructura típica)

```
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score

xgb = XGBClassifier(
    n_estimators=400, # numero de arboles
    max_depth=4, # complejidad de cada arbol
    learning_rate=0.05, # paso: contribucion de cada arbol
```

```
    subsample=0.8, # fraccion de filas por arbol
    colsample_bytree=0.8, # fraccion de columnas por arbol
    reg_lambda=1.0, # regularizacion L2
    random_state=42,
    n_jobs=-1
)

xgb.fit(X_train, y_train)
pred = xgb.predict(X_test)
print("Accuracy XGB:", accuracy_score(y_test, pred))
```

Interpretación práctica:

- `learning_rate` bajo + más `n_estimators` suele generalizar mejor.
- `subsample` y `colsample_bytree` reducen overfitting.
- `max_depth` controla complejidad (profundidad típica: 3–6).

8.4. Random Forest vs XGBoost (decisión práctica)

- Random Forest: gran baseline, poco tuning, robusto.
- XGBoost: suele ganar en rendimiento, pero requiere ajuste cuidadoso.

9. Comparativa global: cuándo usar cada método

Método	Tipo	Escalado	Interpretabilidad	Potencia
Árbol (clasif.)	Clasificación	No	Alta	Media
Árbol (reg.)	Regresión	No	Alta	Media
Multiclase (OvR/OvO)	Clasificación	Depende del base	Media	Media/Alta
KNN	Clasificación/Regresión	Sí	Baja	Media
SVM lineal	Clasificación	Sí	Baja	Alta
Random Forest	Ambos	No	Media	Alta
XGBoost	Ambos	No	Baja	Muy Alta

9.1. Regla rápida de elección

- Necesitas reglas explicables: árbol.
- Necesitas buen baseline robusto: Random Forest.
- Quieres máximo rendimiento en tabular: XGBoost (y tuning).
- Datos muy desbalanceados: métricas (ROC-AUC/PR-AUC) + pesos de clase.
- Método basado en distancia/hiperplano: escalar sí o sí (KNN/SVM).

10. Mini-chuleta de funciones (sklearn) para examen

10.1. Separación train/test

```
train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

Clave: `stratify` en clasificación.

10.2. Entrenar, predecir y evaluar

```
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

- Clasificación: `accuracy_score`, `confusion_matrix`, `classification_report`
- Regresión: `mean_squared_error`, `r2_score` o `model.score`

10.3. ROC-AUC

```
roc_auc_score(y_test, scores_or_proba)
```

Scores válidos:

- Árbol: `predict_proba(X)[:,1]`
- SVM: `decision_function(X)`